



BITS Pilani
Pilani Campus

Genetic Algorithm

Contact Session 6

Artificial and Computational Intelligence — DSE ZG557

Focus: population-based evolutionary search — representation, fitness, selection, crossover and mutation

Akhil P V | BITS Pilani WILP

Reference: T1 §4.1.4 · research papers and web resources

From CS5 to CS6

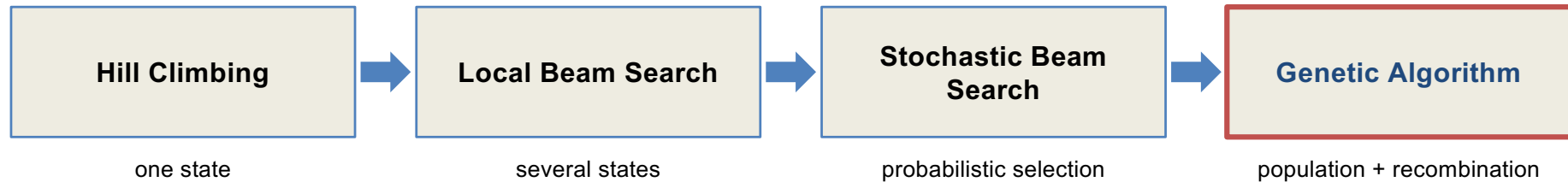


Contact Session 5

Local search improves candidate solutions.

Contact Session 6

Genetic Algorithm evolves a population of candidate solutions.



GA is the natural continuation of stochastic beam search.

T1, §4.1.4.

Learning Objectives



By the end of the session, students should be able to:

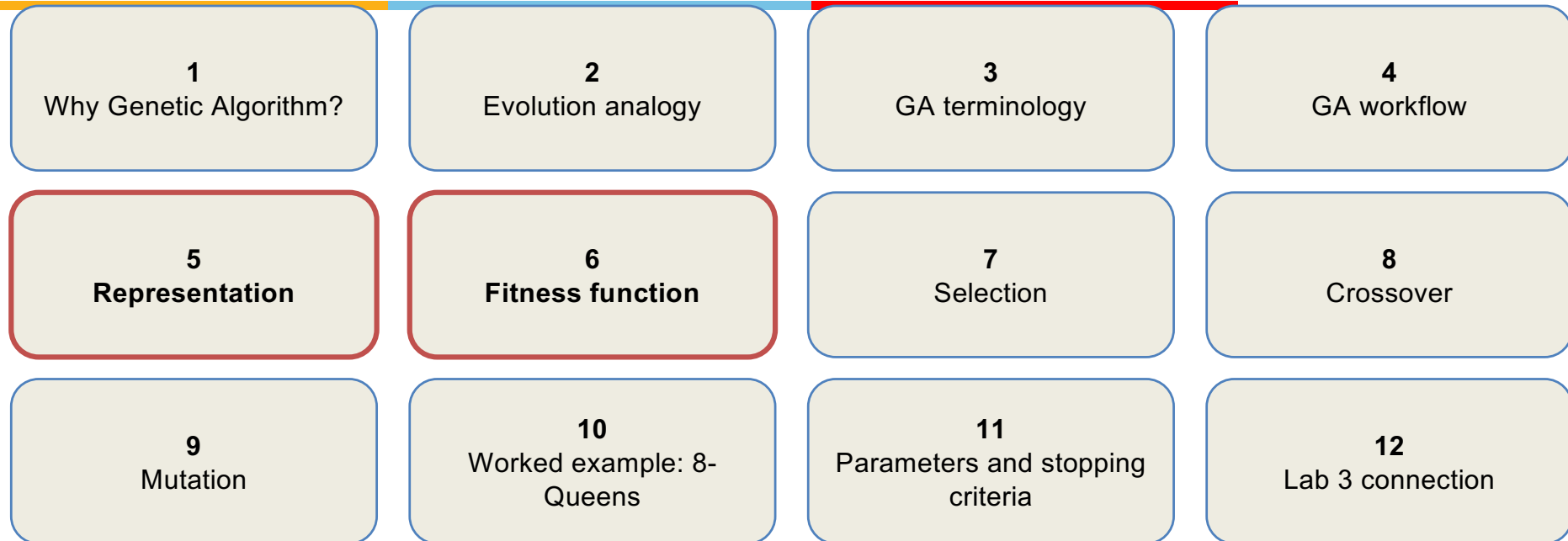
1. Explain the basic idea of Genetic Algorithm.
2. Define population, chromosome, gene and fitness.
3. Represent a solution as a chromosome.
4. Explain selection, crossover and mutation.
5. Trace one generation of a GA.
6. Apply GA to N-Queens or TSP-style optimisation.
7. Discuss parameters, advantages and limitations.
8. Prepare for Lab 3 implementation.

Session focus

Objectives 1–4 build the vocabulary and operators; objectives 5–8 turn them into a working implementation.

T1, §4.1.4, and DSE ZG557 Handout, Contact Session 6.

Session Roadmap



Representation and fitness are the two key design decisions in any GA.

T1, §4.1.4.

Why Do We Need Genetic Algorithms?

Difficult optimisation problems have:

- huge search spaces
- many local optima
- no easy exact method
- complex constraints
- multiple near-good solutions

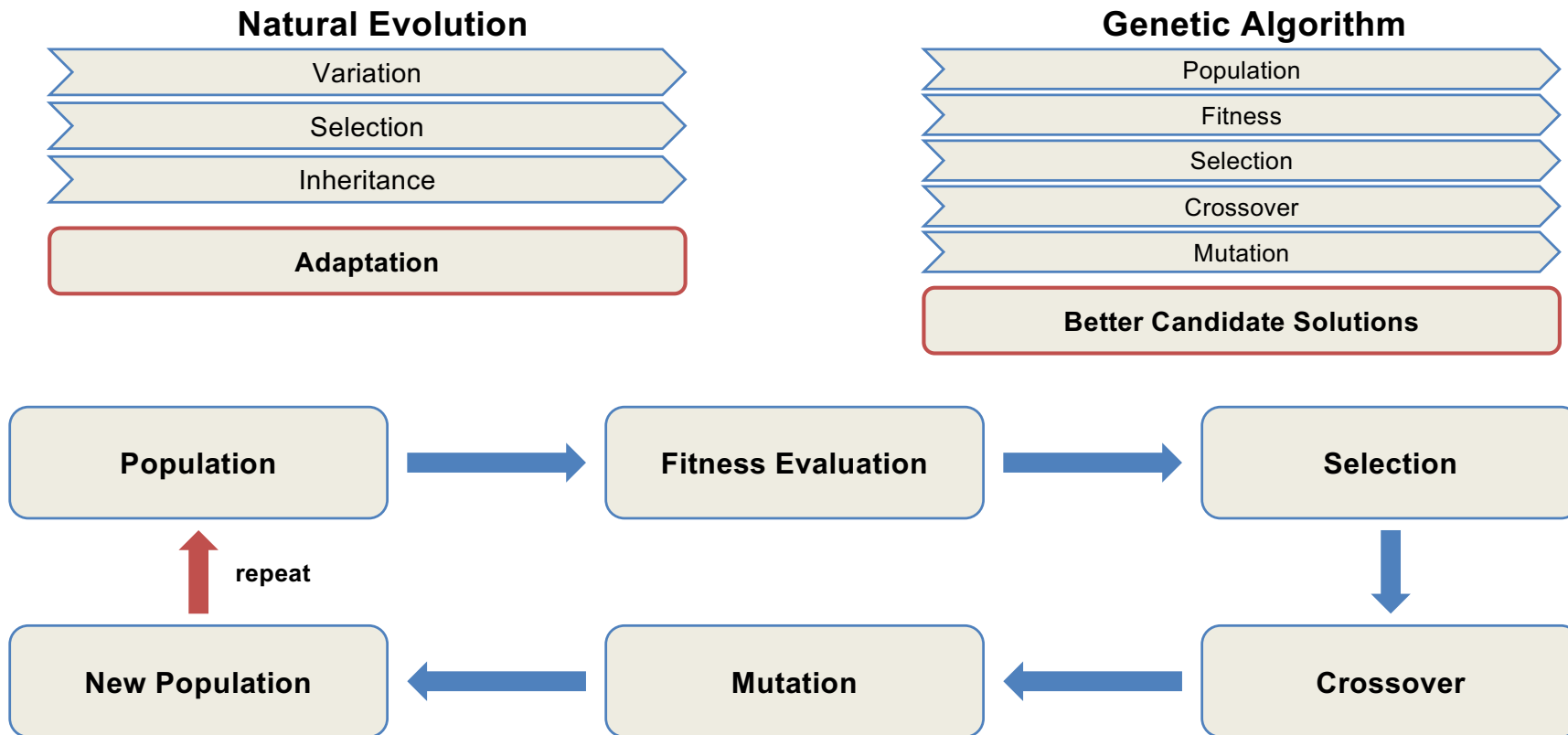
Examples:

- N-Queens
- Travelling Salesman Problem
- job-shop scheduling
- circuit layout
- timetable generation
- feature selection

GA is useful for difficult optimisation problems where exhaustive search is impractical.

T1, §4.1.4.

Basic Idea of Evolutionary Search



The analogy ends here — GA is an algorithm, not a biology lesson.

T1, §4.1.4.

What Is a Genetic Algorithm?

A Genetic Algorithm is a stochastic search method that:

- maintains a population of candidate solutions;
- evaluates each solution using a fitness function;
- selects better candidates more often;
- creates new candidates through crossover and mutation;
- repeats this process over successive generations.

Textbook view

“GA is a variant of stochastic beam search where successor states are generated by combining two parent states.”

GA combines random exploration with fitness-directed improvement.

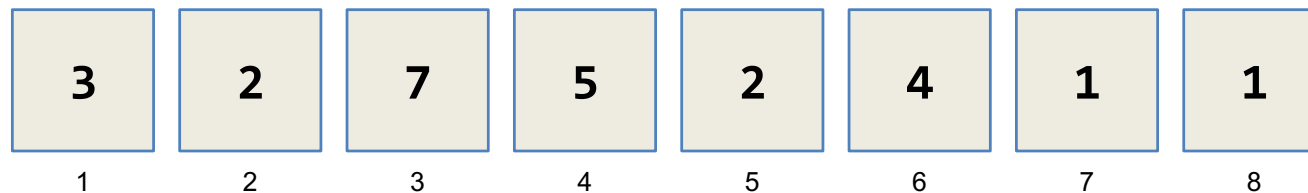
GA is a heuristic search — it does not guarantee the global optimum.

GA Terminology



Term	Meaning
Population	The set of candidate solutions held at one time
Individual / Chromosome	One candidate solution, encoded as a string of values
Gene	One component, or position, of a chromosome
Allele	The value taken by a gene at that position
Fitness	A quality score telling us how good a candidate is
Generation	One complete population-update iteration

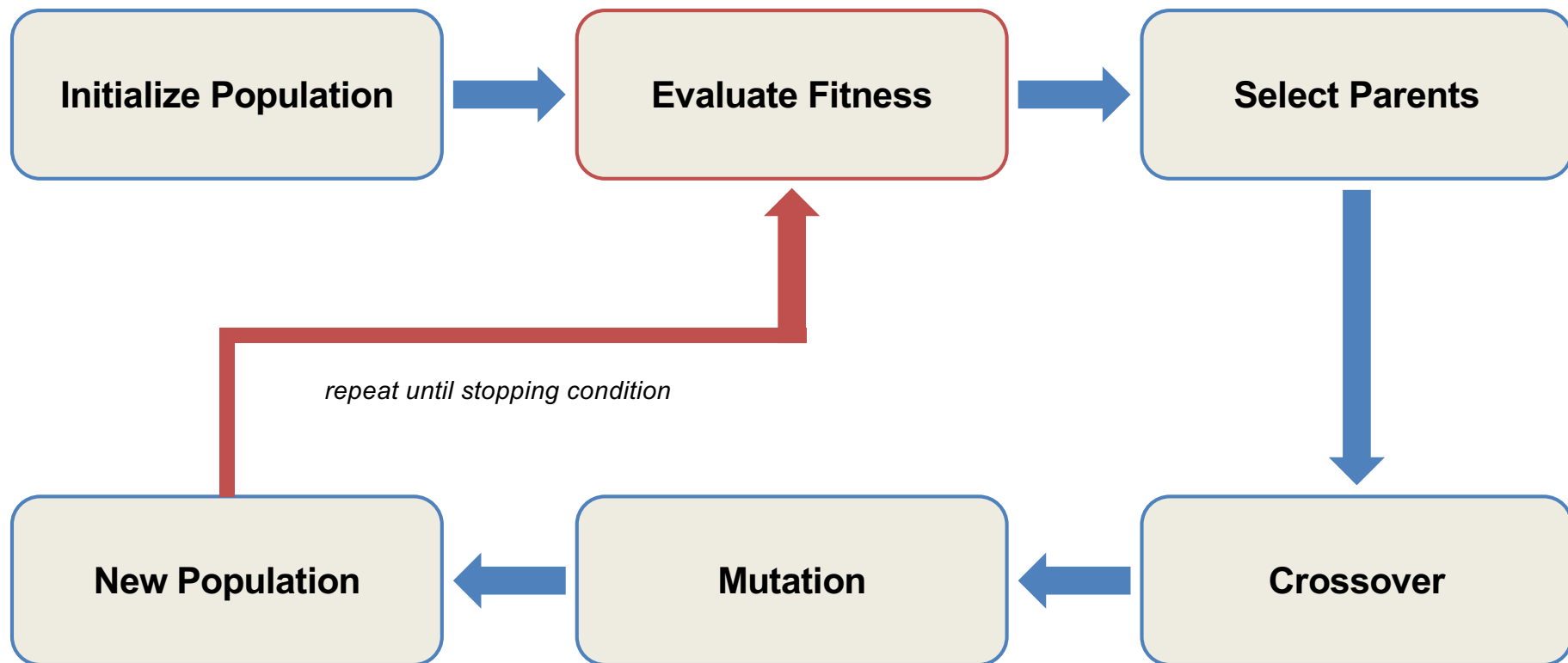
A chromosome of eight genes



Each position = one gene; each value = one allele.

[3,2,7,5,2,4,1,1]

GA Workflow



The rest of the session expands each stage of this workflow.

Generic Genetic Algorithm

```
GENETIC-ALGORITHM(population, fitness_fn)
repeat
  new_population <- empty
  repeat until new_population is full
    x <- SELECT(population, fitness_fn)
    y <- SELECT(population, fitness_fn)
    child <- Crossover(x, y)
    with small probability: child <- MUTATE(child)
    add child to new_population
  population <- new_population
until stopping condition
return the best individual in population
```

Every GA needs selection, crossover, mutation, replacement and termination.

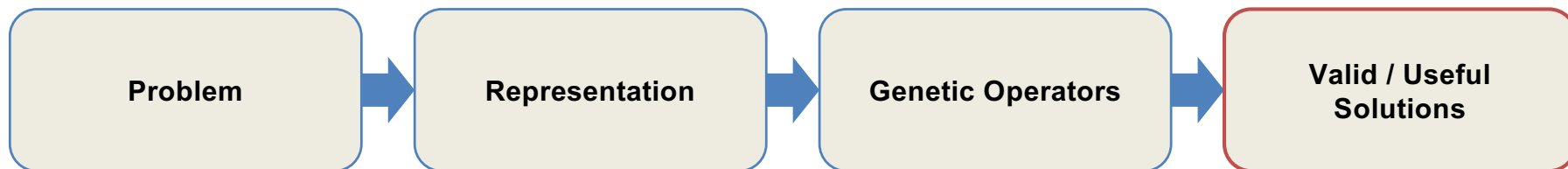
This is pseudocode for understanding, not implementation code.

T1, §4.1.4.

Design Decision 1: Representation

The chromosome representation must answer:

1. What does one individual represent?
2. Does every chromosome represent a valid solution?
3. Can crossover create meaningful children?
4. Can mutation create useful variation?
5. Is fitness easy to compute?



Representation strongly affects GA performance.

T1, §4.1.4.

Representation Example: 8-Queens

	1	2	3	4	5	6	7	8
1							Q	Q
2		Q			Q			
3	Q							
4						Q		
5				Q				
6								
7			Q					
8								

Chromosome

[3,2,7,5,2,4,1,1]

Interpretation

Gene index = column; gene value = row of the queen.

Example: gene 1 = 3 → queen in column 1, row 3.

Advantages

- Always exactly one queen per column
- Compact: eight genes describe a full board
- Easy to mutate: change one gene value

A good representation can build useful constraints into the chromosome.

T1, §4.1.4.

Fitness Function



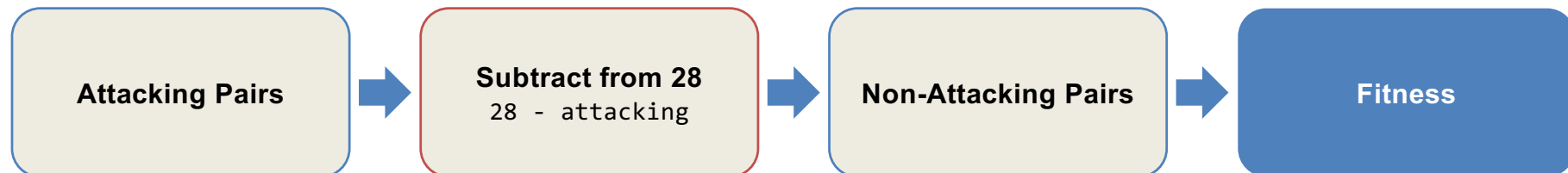
Fitness measures solution quality.

Higher fitness = better individual

For 8-Queens

maximum number of queen pairs = $C(8,2) = 28$

fitness = number of NON-ATTACKING queen pairs



Count the attacking pairs, subtract them from 28, and the remainder is the fitness.

Goal: fitness = 28 — no queen attacks any other.

T1, §4.1.4.

Fitness Example



Worked example

total queen pairs = 28

attacking pairs = 5

fitness = 28 - 5 = 23

Comparing chromosomes (bar length is proportional to fitness out of 28)

Better chromosome — fitness = 26



Our chromosome — fitness = 23



Poorer chromosome — fitness = 11



Ranking: 26 > 23 > 11

Key takeaway: Fitness allows partial progress to be rewarded.

T1, §4.1.4.

Initial Population



Individual	Chromosome	Fitness
A	32752411	24
B	24748552	23
C	32752124	20
D	24415417	11

Population size: $k = 4$

Study note

This population size is for illustration only. Real applications commonly use much larger populations.

- Each individual is an eight-digit chromosome giving one queen's row per column.
- The initial population is generated at random, so fitness values vary widely.

Key takeaway: The population provides multiple starting points in the search space.

T1, §4.1.4.

Selection



Selection chooses the parents for reproduction.

Higher fitness → higher probability of selection

$$P(i) = \text{fitness}(i) / \sum \text{fitness}(j) \quad \text{total} = 78$$

A	24		→ 31%
B	23		→ 29%
C	20		→ 26%
D	11		→ 14%

Selection favours better solutions without removing randomness.

T1, §4.1.4.

Selection Mechanisms

Method	Basic idea
Fitness-Proportionate	Probability proportional to fitness
Tournament Selection	Choose the best from a random subset
Rank Selection	Selection based on fitness rank
Elitism	Preserve the best individual directly

Too much selection pressure

→ loss of diversity, premature convergence.

Too little selection pressure

→ slow progress.

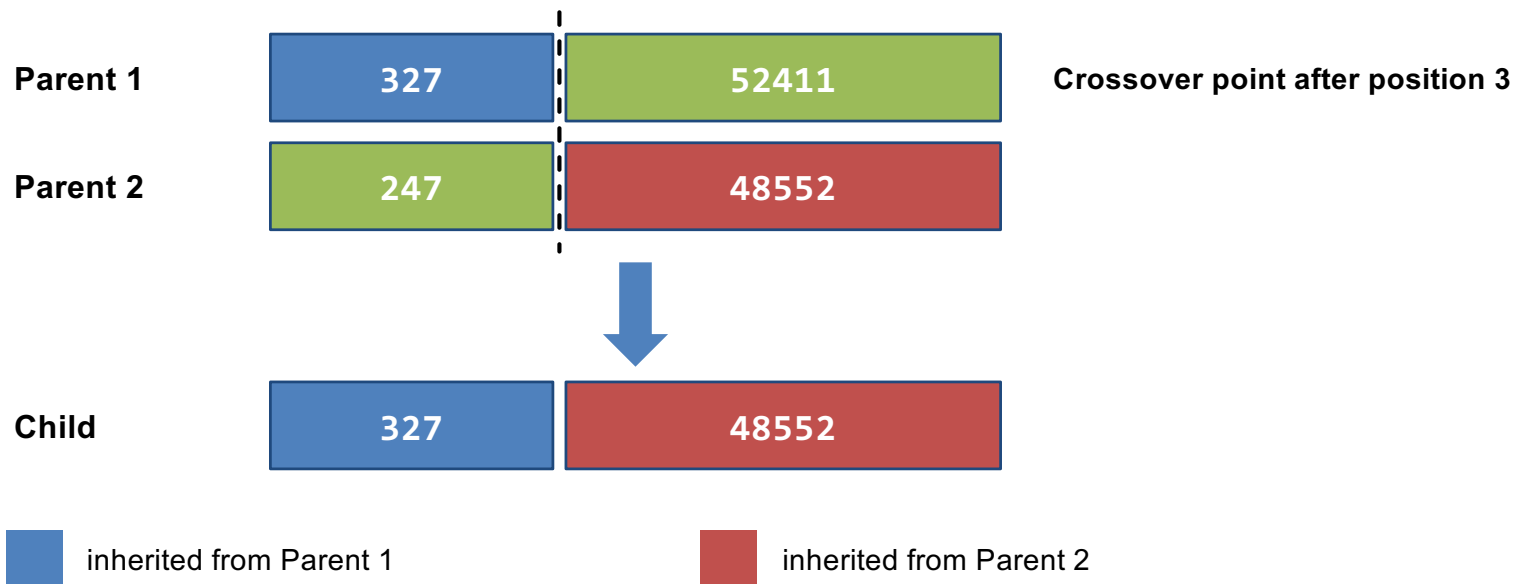
Selection must balance quality and diversity.

T1, §4.1.4.

Crossover



Crossover combines two parent chromosomes to create offspring.



Combine useful components from different parents.

T1, §4.1.4.

Why Crossover Can Help

Suppose a useful partial pattern is:

246*****

- First queen in row 2
- Second queen in row 4
- Third queen in row 6
- Remaining positions unspecified



a schema — a useful partial chromosome pattern

Schema / Schemata

A useful partial chromosome pattern.

Key idea

A GA may combine useful building blocks into stronger solutions.

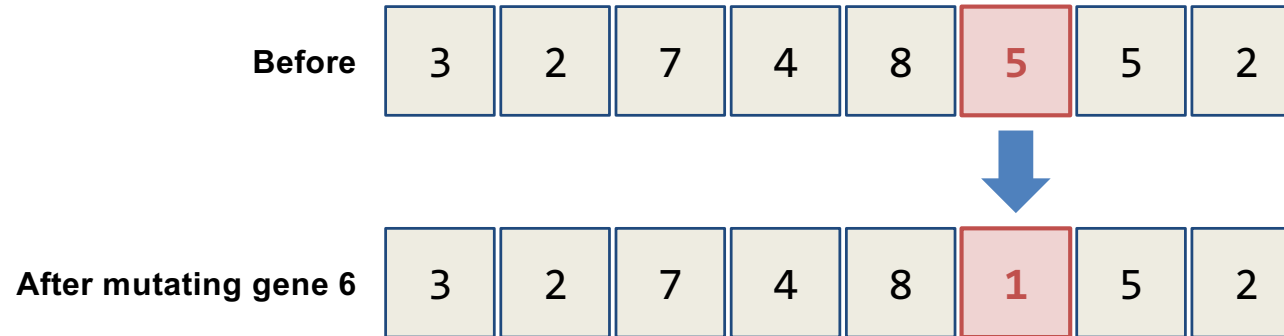
Crossover is useful when chromosome blocks correspond to meaningful solution components.

T1, §4.1.4.

Mutation



Mutation randomly changes a small part of a chromosome.



For 8-Queens, mutation = choose one queen and move it to another row in the same column.

Purpose

- Maintain diversity
- Introduce new genetic material
- Help escape premature convergence

Too much mutation → the search becomes too random.

Too little mutation → diversity may disappear.

Mutation provides controlled random variation.

T1, §4.1.4.

One Full GA Generation



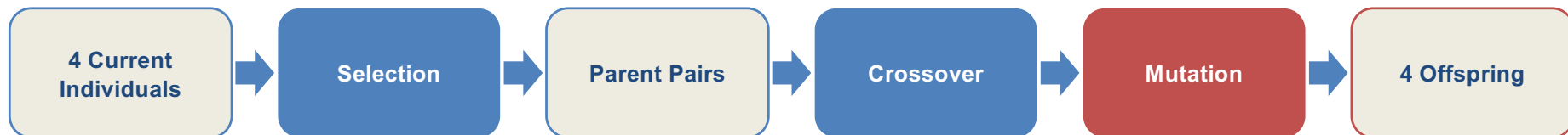
The seven steps of a generation

1. Start with the population.
2. Compute fitness.
3. Select parent pairs.
4. Apply crossover.
5. Apply mutation.
6. Replace the old population.
7. Check the stopping condition.

$$P_t \rightarrow P_{(t+1)}$$

One generation

- Population size stays constant.
- Crossover applies to every selected pair.
- Mutation applies only with a small probability.



A generation transforms one population into the next.

T1, §4.1.4.



Worked Example: 8-Queens Mini Generation

ID	Chromosome	Fitness
A	32752411	24
B	24748552	23
C	32752124	20
D	24415417	11

Selection probabilities

A 31% C 26%

B 29% D 14%

Total fitness = 78. Each share is that individual's fitness divided by 78.

Selected parent pairs: (A,B) and (A,C)

Important — Selection is random but fitness-biased — a high-fitness individual may be selected more than once, and high fitness does not guarantee selection.

High-fitness individuals are more likely to contribute to the next generation.

T1, §4.1.4.

Crossover and Mutation in the Mini Example

Pair 1 — cut after position 3

A	3	2	7	5	2	4	1	1
B	2	4	7	4	8	5	5	2
Child 1	3	2	7	4	8	5	5	2

Pair 2 — cut after position 5

A	3	2	7	5	2	4	1	1
C	3	2	7	5	2	1	2	4
Child 2	3	2	7	5	2	1	2	4

Mutation step — applied to Child 2

Child 2	3	2	7	5	2	1	2	4
	3	2	7	1	2	1	2	4

One gene at position 4 changes: 5 → 1.

from first parent
 from second parent
 mutated gene

GA creates candidates through recombination followed by possible variation.

Stopping Conditions

A GA may stop when:

- a target fitness value has been reached;
- a maximum number of generations has been reached;
- a time limit has been reached;
- fitness has not improved for many generations;
- the population has converged.

For 8-Queens: fitness = 28 → a solution has been found

A GA needs an explicit, practical termination rule.

Parameter Choices



The main parameters of a GA and what each one controls

Parameter	Typical effect
Population size	Diversity against computational cost
Crossover probability	How often recombination occurs
Mutation probability	Exploration and diversity
Selection pressure	Speed against premature convergence
Number of generations	Runtime and the opportunity to improve

There is no universally best GA parameter setting.

Premature Convergence

Premature convergence — the population becomes too similar too early.

Diverse Population



Increasing Similarity



Premature Convergence

Loss of diversity → the search gets stuck

Possible causes

- excessive selection pressure
- too little mutation
- a small population
- poor representation

Possible responses

- increase mutation slightly
- increase the population
- preserve diversity
- restart
- adjust the selection method or pressure

A GA needs diversity in order to keep exploring.

Encoding Matters



8-Queens

[3, 2, 7, 5, 2, 4, 1, 1]

One queen per column.

Gene index = column, gene value = row.

Travelling Salesman

[A, C, D, B, E]

Visit the cities in that order.

The chromosome is a permutation.

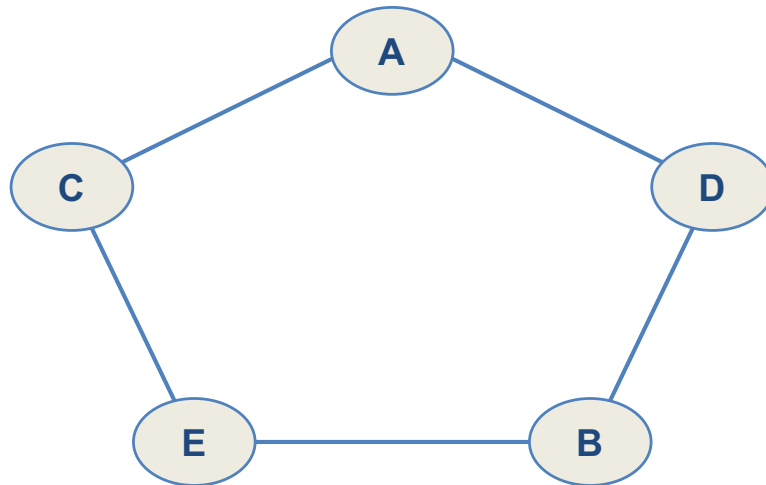
Warning

Ordinary one-point crossover can create invalid TSP tours — repeated cities or missing cities.

Representation and genetic operators must fit the structure of the problem.

T1, §4.1.4.

GA for the Travelling Salesman Problem



[A,D,B,E,C]

The closed tour above, written as a chromosome.

Possible fitness formulations

$\text{fitness} = 1 / (1 + \text{tour length})$

$\text{fitness} = -\text{tour length}$

Both reward shorter tours; the choice depends on the implementation.

Main challenge: crossover and mutation must preserve a valid permutation.

T1, §4.1.4.

GA vs Hill Climbing



Feature	Hill Climbing	Genetic Algorithm
Search units	One current state	A population
Move generation	Local neighbour	Crossover and mutation
Randomness	Optional, variant-dependent	Central
Diversity	Low	Higher
Memory	Very low	Population memory
Main risk	Local optimum	Premature convergence

GA is population-based stochastic local search.

Neither method is universally better — the choice depends on the problem.

T1, §4.1.4.

Advantages of Genetic Algorithms

Advantages

- Works with large search spaces
- Requires no gradient information
- Handles discrete and combinatorial problems
- Explores several regions at once
- Can combine useful solution components
- Conceptually parallelisable

Applications

- Circuit layout
- Job-shop scheduling
- Travelling Salesman Problem
- Feature selection
- Engineering design

GA is flexible across many difficult optimisation tasks.

Limitations of Genetic Algorithms

Limitations

- No guaranteed optimal solution in normal practical use
- Premature convergence
- Fitness evaluation can be expensive
- Representation design can be difficult
- Parameter tuning is problem-dependent
- A poorly chosen crossover may be ineffective
- Results vary because of randomness

Caution GA is not automatically better than simpler local-search methods.

GA success depends on careful problem-specific design.

Lab 3: Genetic Algorithm



Expected implementation components

1. Problem representation
2. Initial population
3. Fitness function
4. Parent selection
5. Crossover
6. Mutation
7. Replacement strategy
8. Stopping condition
9. Output the best solution
10. Report performance

Suggested problems

- N-Queens
- Travelling Salesman Problem
- A small scheduling or timetable problem

Lab 3 implements the complete GA cycle.

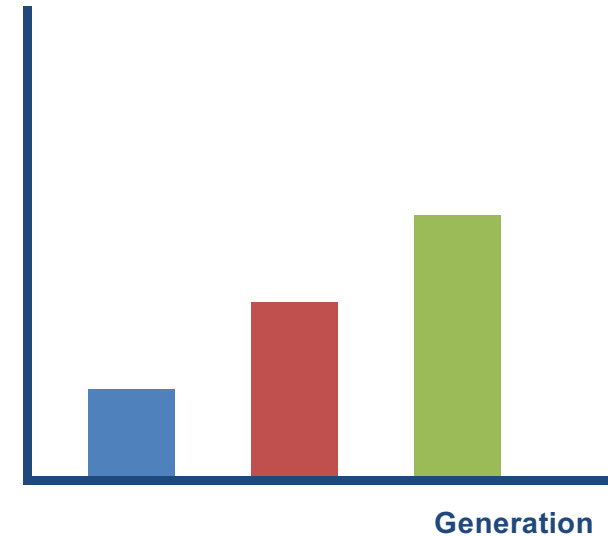
Lab 3: What to Measure



Record for every run

- Population size
- Crossover probability
- Mutation probability
- Number of generations
- Best fitness per generation
- The final solution
- Success or failure
- Runtime
- The effect of parameter changes

Best Fitness



Best fitness should rise, then flatten as the population converges.

Do not report only one final solution — analyse GA behaviour across generations.

Mini Activity: Design a GA



Design a Genetic Algorithm for N-Queens.

5 minutes

Answer these seven questions in your group

1. What is the chromosome?
2. What is one gene?
3. What is the fitness function?
4. How will parents be selected?

5. What crossover will be used?
6. What mutation will be used?
7. When will the algorithm stop?

T1, §4.1.4.

Common Mistakes

Avoid

1. Confusing fitness with cost.
2. Using lower-is-better values without adjusting the selection logic.
3. Using a crossover that creates invalid solutions.
4. A mutation probability that is too high or too low.
5. No stopping condition.
6. No best-solution tracking or elitism.
7. Reporting only the final output.
8. Assuming a GA always finds the global optimum.

Memory aid: the GA cycle



T1, §4.1.4.

Exam-Focused Summary



Must be able to state

- A GA is a population-based stochastic search inspired by natural selection.
- The five-stage cycle: Initialise, Evaluate, Select, Crossover, Mutate.
- Fitness measures solution quality, not distance to a goal.
- Selection is proportional to fitness.
- Crossover recombines; mutation diversifies.

Must be able to do

- Encode 8-Queens as an 8-digit string such as 32752411 (index = column, value = row).
- Compute fitness as the number of non-attacking pairs; maximum $C(8, 2) = 28$.
- Convert fitness values into selection percentages.
- Perform a single-point crossover by hand.
- Apply and describe one mutation.
- State one stopping condition.

Exit Check



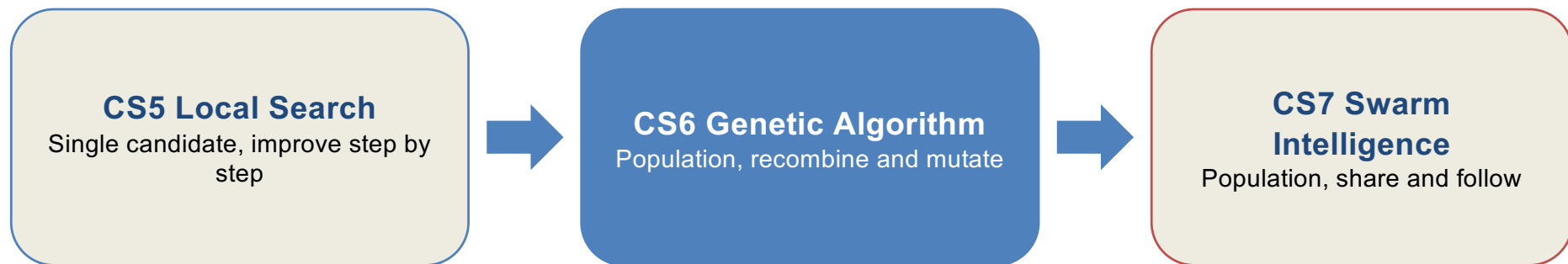
1. What does a chromosome represent in a GA?
2. Why is fitness maximised rather than minimised here?
3. What is the maximum fitness for 8-Queens, and why?
4. What does single-point crossover actually exchange?
5. Why is mutation still needed if crossover exists?
6. What is premature convergence, and one way to reduce it?
7. Why is a permutation representation required for TSP?
8. Does a GA guarantee an optimal solution? Justify.

Answers discussed in session — attempt before Lab 3.

T1, §4.1.4, Genetic Algorithms.

Bridge to CS7: Swarm Intelligence

- CS6 evolved a population through selection, crossover and mutation.
- CS7 keeps the population, but replaces recombination with social interaction: individuals share information and adapt their movement (particle swarm, ant colony).



Carry forward: representation, fitness and stopping conditions remain the same design decisions.

References



- **T1:** Russell, S. & Norvig, P. — Artificial Intelligence: A Modern Approach, §4.1.4 Genetic Algorithms (primary source for this session).
- **Course handout:** DSE ZG557 — Artificial and Computational Intelligence, Contact Session 6.
- **Lab 3:** Genetic Algorithm implementation — N-Queens and TSP (LO3).
- **Supplementary:** Goldberg, D. E. — Genetic Algorithms in Search, Optimization and Machine Learning.

T1, §4.1.4, Genetic Algorithms.